SmartLogix – Plataforma Inteligente para la Gestión Logística de eCommerce.

Diseño de Arquitectura y Patrones de Microservicios.

Sección: 003D.

Docente: Claudio Esteban Rojas Rodriguez.

Integrantes:

- **Gabriel Jeria.**
- **Vicente Palacios.**
- **Isak Chacana.**

Introducción.

Contexto.

El auge del comercio electrónico ha impulsado la necesidad de sistemas más eficientes y escalables para la gestión de inventarios, pedidos y envíos. SmartLogix es una startup en crecimiento que busca ofrecer una plataforma moderna e integrada para la optimización de procesos logísticos en pequeñas y medianas empresas (PYMEs) de eCommerce. La empresa ha identificado que muchas de estas PYMEs aún dependen de sistemas monolíticos y procesos manuales, lo que genera inconsistencias en la información, retrasos en la entrega y dificultades para manejar picos de demanda.

Para resolver estos problemas, SmartLogix quiere desarrollar una solución basada en microservicios, con una capa frontend moderna y flexible, y un backend escalable y seguro, permitiendo a las PYMEs gestionar su operación logística con mayor eficiencia y reducir costos operativos.

Actualmente, las empresas que utilizan soluciones logísticas tradicionales enfrentan desafíos como sincronización deficiente de inventarios, errores en la asignación de pedidos, y demoras en la coordinación de envíos. Muchos de estos problemas surgen porque sus sistemas monolíticos no permiten la automatización ni integración con nuevas tecnologías. SmartLogix ha decidido desarrollar una arquitectura moderna basada en microservicios para garantizar la escalabilidad y flexibilidad de su plataforma, facilitando la automatización de la gestión logística.

Desarrollo

Arquitectura

Para este proyecto utilizaremos la arquitectura de microservicios, además de siendo el patrón de arquitectura pedido por el cliente, este nos permitirá escalar y flexibilizar nuestra arquitectura, permitiendo al negocio expandir su base de PYMEs sin tener que preocuparse de la capacidad del software usado, en contraste a una infraestructura monolítica que no permitiría ese nivel de escalado y requeriría mayor

mantenimiento cada vez que se quiera aumentar la cantidad de PYMEs siendo servidas.

Requerimientos funcionales

1. Registro de usuarios: El sistema debe permitir al administrador registrar nuevos usuarios en la plataforma, ingresando nombre, correo electrónico, contraseña y rol.

2. Validación de usuario duplicado: El sistema debe validar que el correo electrónico ingresado en el registro no exista previamente en la base de datos.

3. Almacenamiento seguro de contraseñas: El sistema debe almacenar las contraseñas de los usuarios de forma encriptada antes de guardarlas en la base de datos.

4. Inicio de sesión: El sistema debe permitir que los usuarios inicien sesión mediante correo electrónico y contraseña.

5. Validación de credenciales: El sistema debe verificar las credenciales ingresadas por el usuario contra los datos almacenados en la base de datos de autenticación.

6. Generación de token de acceso: El sistema debe generar un token JWT cuando el usuario inicie sesión correctamente.

7. Gestión de roles: El sistema debe permitir al administrador asignar y modificar roles de usuario dentro de la plataforma.

8. Control de permisos por rol: El sistema debe restringir el acceso a funcionalidades según el rol del usuario: administrador, operador logístico, supervisor y transportista.

9. Creación de productos: El sistema debe permitir al administrador registrar productos en el módulo de inventario.

10. Almacenamiento de productos: El sistema debe guardar los productos creados en la base de datos del microservicio de inventario.

11. Confirmación de creación de producto: El sistema debe mostrar una confirmación cuando un producto haya sido creado exitosamente.

12. Consulta de stock: El sistema debe permitir que el operador logístico y el supervisor consulte el stock disponible de los productos.

13. Visualización detallada de stock: El sistema debe mostrar el stock total y el stock por bodega de cada producto consultado.

14. Filtro de búsqueda de productos: El sistema debe permitir filtrar la consulta de stock por producto.

Requerimientos no funcionales

1. Seguridad de autenticación: El sistema debe proteger el acceso mediante autenticación segura basada en JWT y manejo de sesión segura.

2. Protección de credenciales: Las contraseñas no deben almacenarse en texto plano y deben mantenerse cifradas en la base de datos.

3. Protección de sesión: El sistema debe evitar la exposición del token JWT en el frontend, utilizando mecanismos seguros como cookies o manejo interno por BFF.

4. Control de acceso: El sistema debe aplicar autorización basada en roles para restringir el acceso a funciones según el perfil del usuario.

5. Comunicación segura: Toda la comunicación entre cliente, frontend, gateway y servicios debe realizarse mediante HTTPS.

6. Escalabilidad: La arquitectura debe permitir escalar los microservicios de forma independiente según la demanda.

7. Disponibilidad: El sistema debe estar diseñado para mantener continuidad operativa ante aumentos de carga o fallos parciales de servicios.

8. Mantenibilidad: El sistema debe permitir modificaciones o incorporación de nuevas funcionalidades sin afectar significativamente a los demás módulos.

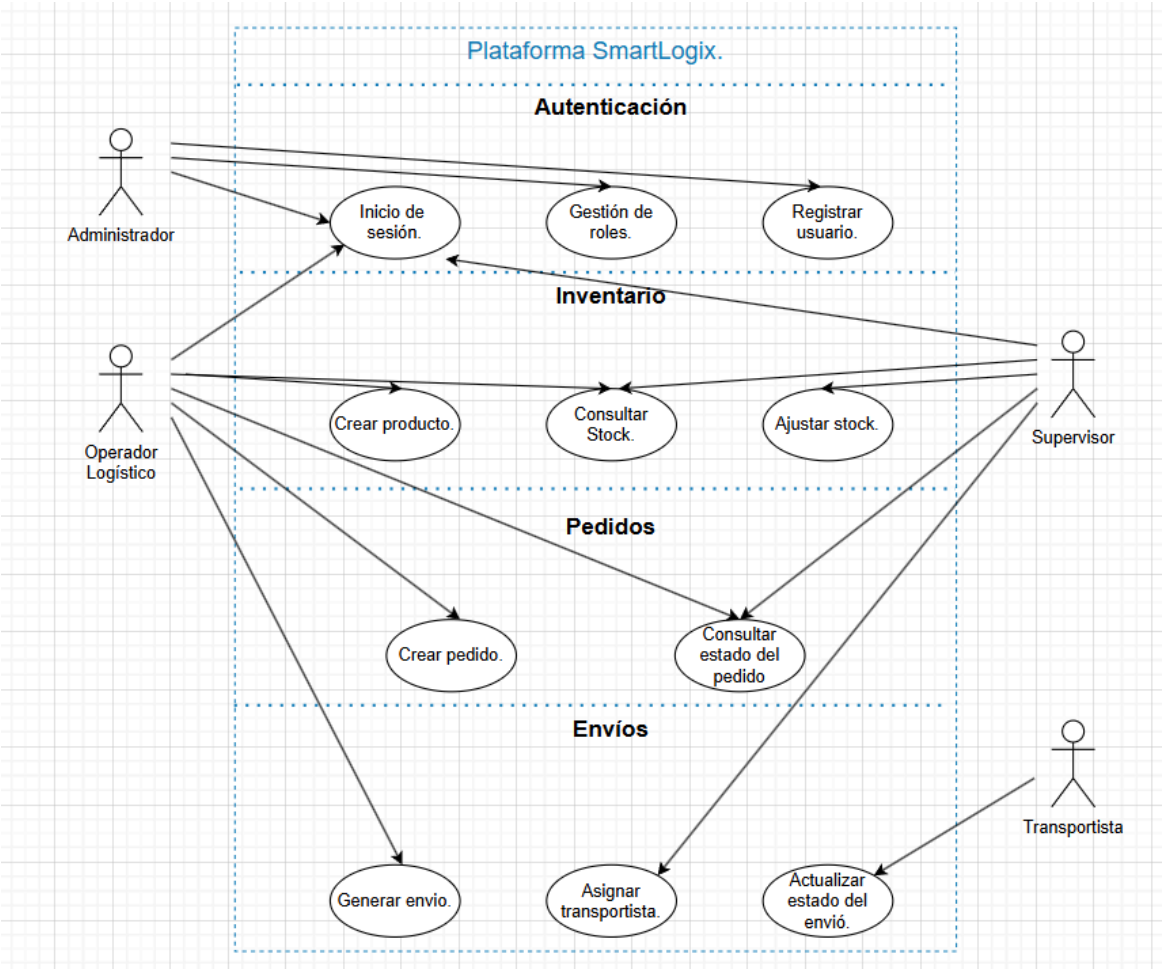
9. Rendimiento: El sistema debe responder de manera eficiente a consultas de inventario, pedidos y envíos, incluso ante múltiples solicitudes concurrentes.

10. Persistencia de datos: Cada microservicio debe contar con su propia base de datos para asegurar independencia y consistencia en la gestión de la información.

Con esto considerado, tras un análisis de las necesidades del cliente, identificamos 4 microservicios principales que realizaremos:

- **Autentificación (BD)**: Gestionar el inicio de sesión, registro de usuarios.
- **Gestión de Inventario (BD):** Mantiene actualizados los niveles de stock en tiempo real, optimizando la sincronización entre múltiples bodegas y tiendas.
- **Procesamiento de Pedidos (BD):** Automatiza la validación, aprobación y asignación de pedidos, asegurando trazabilidad y reduciendo errores.
- **Coordinación de Envíos (BD):** Mejora la planificación de rutas y la comunicación con transportistas, garantizando tiempos de entrega eficientes.

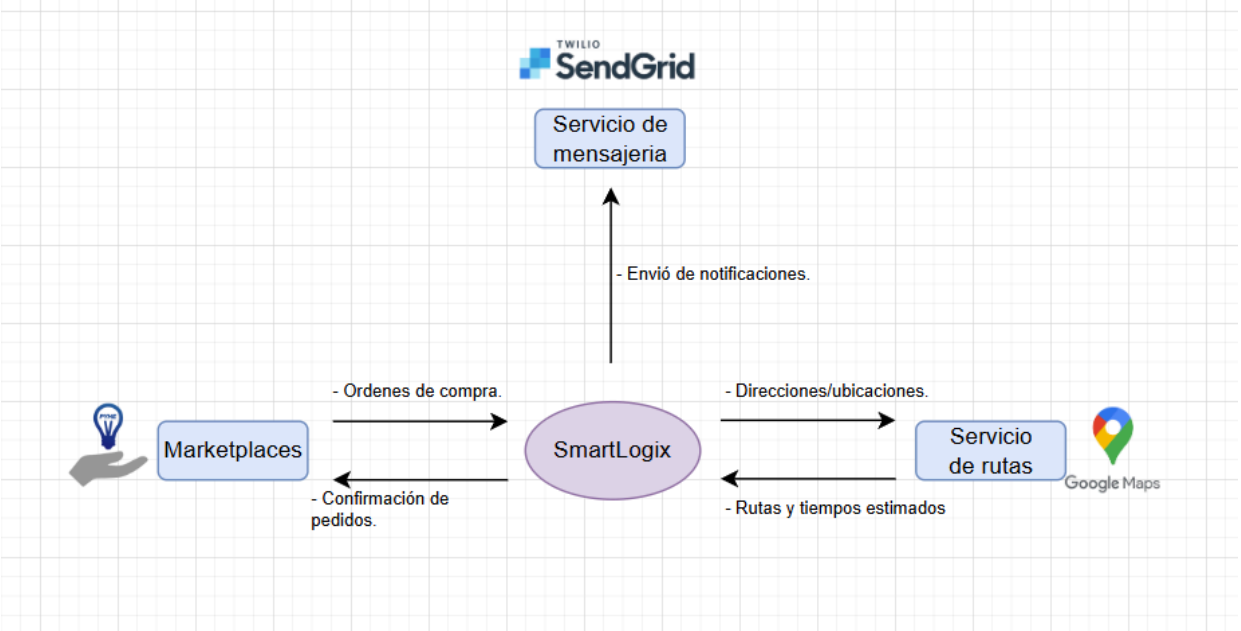
Diagrama de Caso de Uso



Con los microservicios identificados podemos producir las interacciones entre los distintos actores usando nuestro programa, nótese de que el rol de supervisor y

operador logístico pueden ser combinados si es necesario simplificar el sistema.Diagrama de contexto

Con los microservicios ya establecidos se realizó un diagrama de contexto identificando los servicios externos que utilizaremos dentro del proyecto.

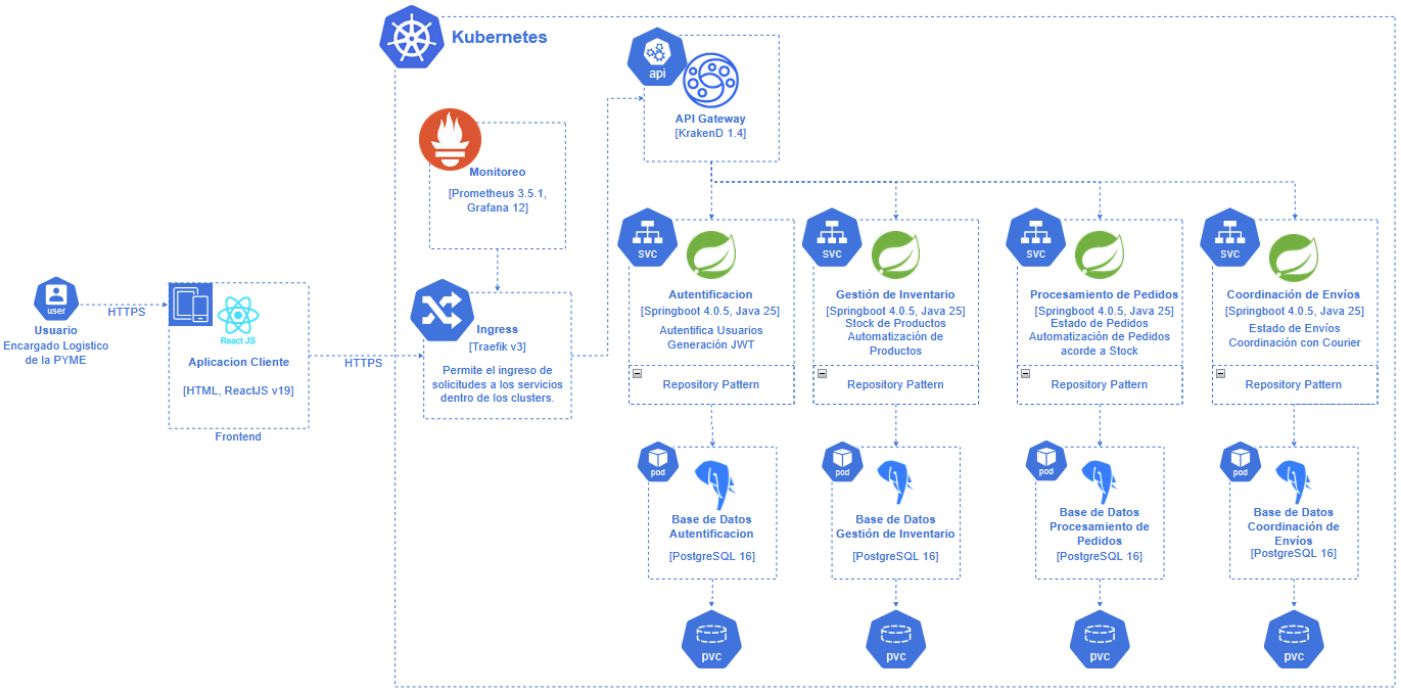


1. Google Maps: Se utilizará Google Maps como servicio principal para la gestión de rutas. Este permitirá Entregar direcciones y ubicaciones precisas. Calcular rutas óptimas para los transportistas. Proporcionar tiempos estimados de llegada, mejorando la planificación logística.

2. SendGrid: Se utilizará SendGrid como servicio de mensajería, con el objetivo de mantener una comunicación eficiente con los transportistas. Sus principales funciones serán el envío de notificaciones, garantizar la confiabilidad y escalabilidad en el envío de correos electrónicos.

3. Marketplaces: Los marketplaces externos serán la fuente principal de órdenes de compra, la integración con estos servicios permitirá recepción automática de pedidos generados en las plataformas e-commerce. Procesamiento interno de órdenes para su preparación y despacho. Devolución de confirmaciones de pedidos hacia los marketplaces, asegurando trazabilidad y transparencia en el proceso.

Diagrama de contenedores



Con el objetivo de representar de manera clara la estructura interna del sistema, se diseñó un diagrama de contenedores que muestra los distintos componentes tecnológicos y cómo interactúan entre sí dentro de la plataforma SmartLogix.

La solución se basa en una arquitectura de microservicios desplegada sobre Kubernetes, lo que permite una alta escalabilidad, disponibilidad y desacoplamiento entre los distintos módulos del sistema. Cada microservicio cumple una función específica del negocio, lo que facilita su mantenimiento, evolución y despliegue independiente.

El flujo de comunicación inicia desde el cliente o frontend, desarrollado con ReactJS, el cual se comunica mediante HTTPS hacia el clúster. Las solicitudes ingresan a través de un Ingress Controller permitiendo enrutar el tráfico hacia los servicios internos.

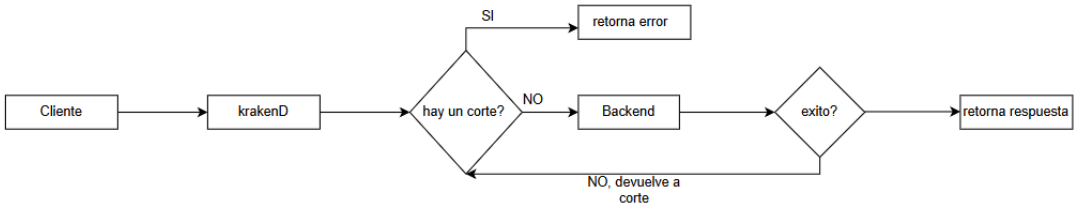
Luego las peticiones pasan por el API Gateway (KrakenD), que centraliza el acceso a los microservicios, gestionando el enrutamiento, seguridad y control de acceso. Esto evita la exposición directa de los servicios internos y mejora la protección del sistema.

Dentro del clúster, se encuentran los principales microservicios autenticación, gestión de inventario, procesamiento de pedidos y coordinación de envíos. Cada microservicio cuenta con su propia base de datos independiente (PostgreSQL), siguiendo el principio de desacoplamiento de datos, lo que mejora la resiliencia y evita dependencias entre módulos.

La arquitectura incorpora herramientas de monitoreo como Prometheus (un sistema de monitoreo open source) y Grafana (un sistema de visualización de data que cuenta con una versión open source y una versión Enterprise con capa gratuita), permitiendo supervisar el estado del sistema, métricas de rendimiento y disponibilidad de los servicios en tiempo real.

Patrones de Diseño

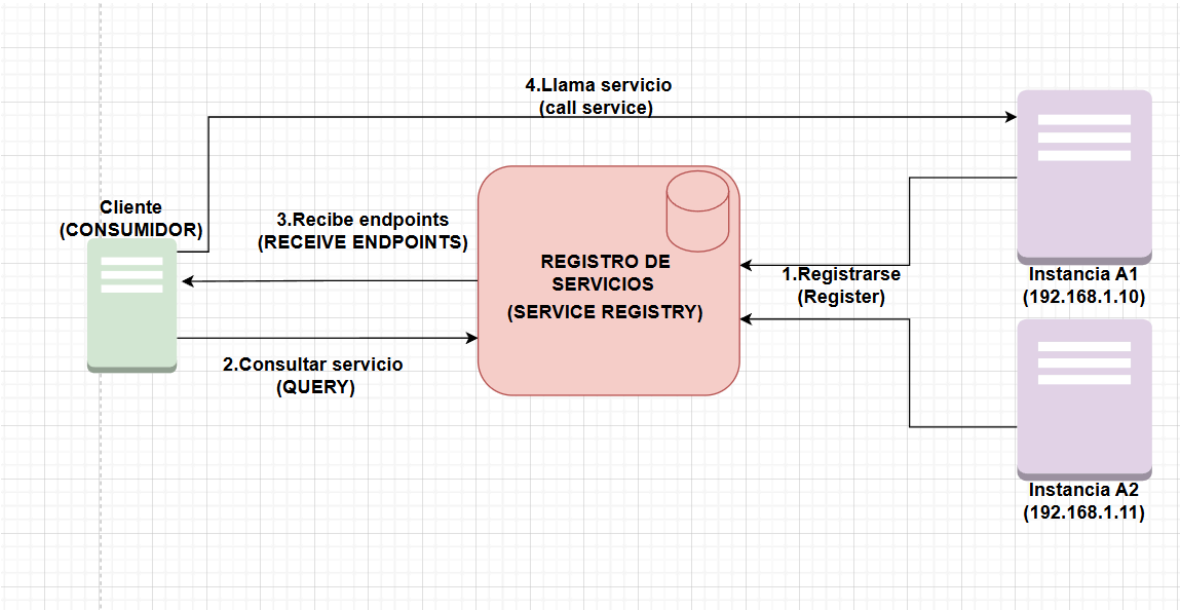
Circuit Breaker



El sistema implementa el patrón Circuit Breaker a nivel de infraestructura, específicamente en el API Gateway (KrakenD), con el objetivo de evitar la sobrecarga de los microservicios ante fallos recurrentes. Este mecanismo permite detectar cuando un servicio backend presenta errores consecutivos y, en lugar de seguir enviando solicitudes, el gateway interrumpe temporalmente la comunicación retornando un error controlado al cliente.

Con esto evitamos que se sature el sistema, mejora la disponibilidad y se protege la infraestructura. Cuando el servicio se recupera, el circuito se cierra automáticamente y se restablece la comunicación.

Service Discovery en Kubernet



En esta arquitectura, el descubrimiento de servicios (**Service Discovery**) es gestionado de forma nativa por Kubernetes, eliminando la necesidad de implementar un registro de servicios manual. Kubernetes proporciona este mecanismo mediante el uso de Services **(Cluster IP)** y un sistema de DNS interno, lo que permite que los microservicios se comuniquen entre sí utilizando nombres lógicos en lugar de direcciones IP.

Cuando un microservicio necesita comunicarse con otro, no requiere conocer su ubicación exacta, sino que realiza la solicitud utilizando el nombre del servicio (por ejemplo, inventario-service). Kubernetes se encarga de resolver este nombre hacia una o varias instancias disponibles (**pods**), distribuyendo la carga de manera automática.

Este enfoque permite una abstracción de la infraestructura adyacente, escalabilidad dinámica de los servicios, balanceo de carga y una alta disponibilidad ante fallos. De esta manera, el sistema mantiene una comunicación eficiente y desacoplada entre microservicios.

Implementación de BBF, JWT y API Gateway

Para mitigar los riesgos de seguridad derivados de la exposición de múltiples microservicios, se implementa una estrategia basada en el uso combinado de un **Backend for Frontend (BFF)**, **tokens JWT** y un **API Gateway (KrakenD)**.

El **BFF** actúa como intermediario entre el cliente y los servicios backend, gestionando de forma segura los tokens de autenticación. En lugar de exponer el JWT al frontend, el BFF almacena el token y entrega al cliente una cookie segura, reduciendo el riesgo de ataques.

Por otro lado el **API Gateway (KrakenD)** funciona como punto único de entrada al sistema, encargándose de enrutar las solicitudes hacia los microservicios correspondientes, aplicando validaciones de seguridad y asegurando que solo las peticiones autenticadas sean procesadas.

Conclusión

En conclusión, la arquitectura propuesta para SmartLogix permite construir una plataforma moderna, escalable y segura, capaz de responder a las necesidades actuales de las PYMEs en el ámbito del comercio electrónico. La adopción de una arquitectura basada en microservicios, desplegada sobre Kubernetes, facilita la separación de responsabilidades, el escalamiento independiente de cada módulo y una mayor flexibilidad ante cambios del negocio.

Además, la incorporación de mecanismos como Service Discovery, API Gateway, BFF, JWT y Circuit Breaker fortalece aspectos críticos como la comunicación interna, la seguridad y la resiliencia del sistema. Estas decisiones permiten no solo proteger los servicios frente a accesos no autorizados, sino también garantizar la disponibilidad y estabilidad de la plataforma ante fallos o aumentos de carga.

En conjunto, la solución diseñada no solo optimiza la gestión de inventarios, pedidos y envíos, sino que también establece una base tecnológica robusta, preparada para integrarse con servicios externos y adaptarse al crecimiento futuro de la empresa.